

LAPORAN PRAKTIKUM
INTERNET OF THINGS (IOT) II

MODUL 3



Disusun Oleh :

Nama : Naila Rahma Chintya Aulya

NPM : 25782020

Kelas : 3A

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, A.Md

JURUSAN TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
POLITEKNIK NEGERI LAMPUNG

2026/2027

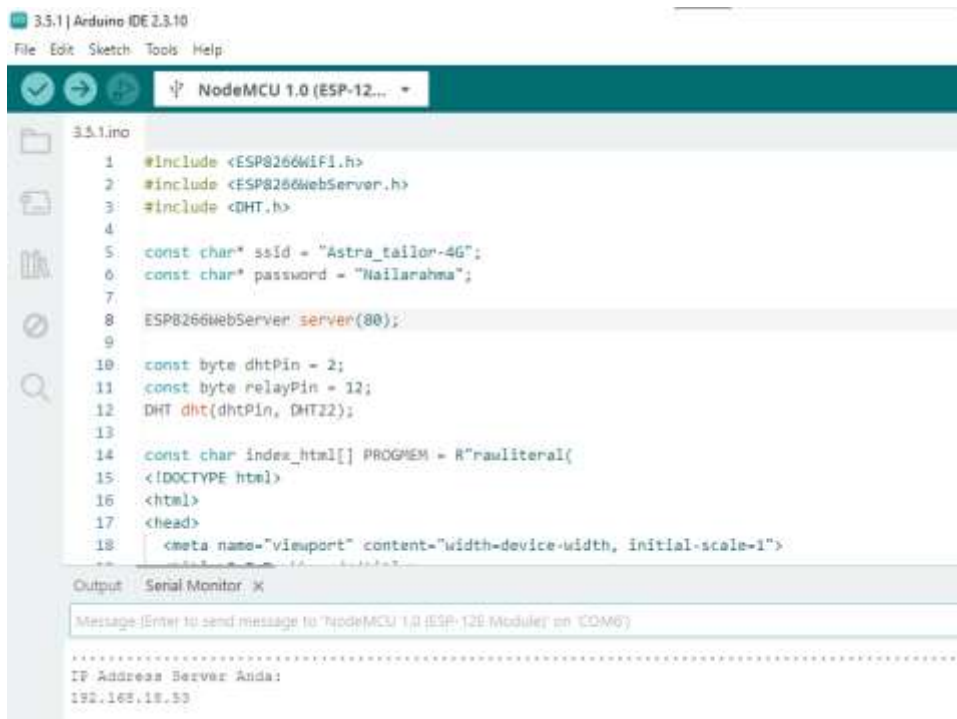
3.5.1 Web Server ESP8266

Pada praktikum 3.5.1, ESP8266 digunakan sebagai **Web Server** yang dapat diakses melalui browser menggunakan alamat IP yang diperoleh setelah ESP8266 terhubung ke jaringan Wi-Fi. Sensor yang digunakan adalah DHT22 untuk membaca suhu, sedangkan LED digunakan sebagai output yang dapat dikendalikan melalui halaman web.

Pada program, ESP8266 terlebih dahulu dihubungkan ke jaringan Wi-Fi menggunakan mode **Station (STA)**. Setelah berhasil terhubung, ESP8266 mendapatkan alamat IP yang kemudian ditampilkan pada Serial Monitor. Alamat IP tersebut digunakan untuk membuka halaman Web Server melalui browser.

DHT22 dihubungkan ke pin D4/GPIO2 untuk membaca suhu. Nilai suhu yang diperoleh kemudian ditampilkan pada halaman web. Selain menampilkan data sensor, halaman web juga menyediakan tombol untuk mengontrol LED. Ketika tombol ON dipilih, ESP8266 akan memberikan logika HIGH pada pin LED sehingga LED menyala. Sebaliknya, ketika tombol OFF dipilih, pin LED diberikan logika LOW sehingga LED mati.

Dengan demikian, pada praktikum ini ESP8266 dapat berfungsi sebagai Web Server yang menampilkan data sensor sekaligus mengendalikan output melalui browser.



```
3.5.1 | Arduino IDE 2.3.10
File Edit Sketch Tools Help

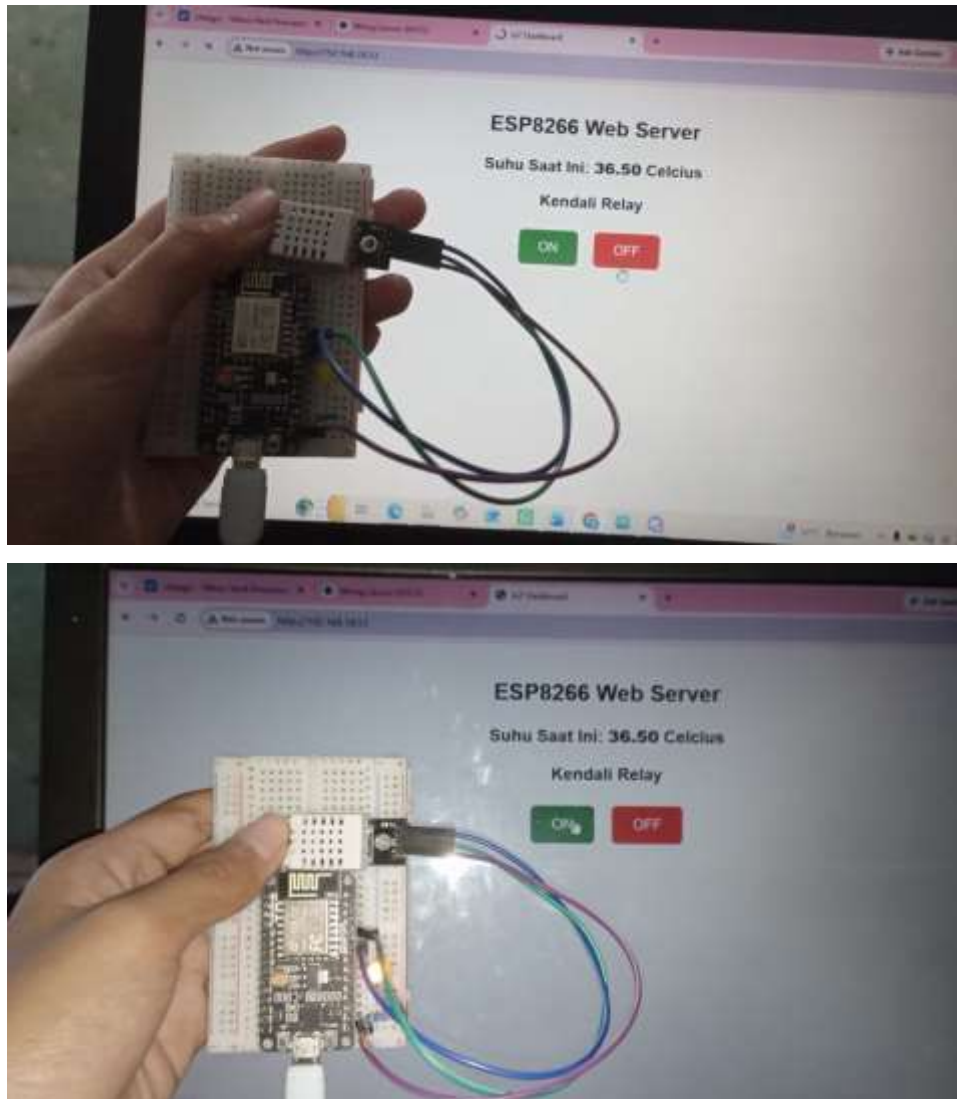
NodeMCU 1.0 (ESP-12...)

3.5.1.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "Astra_tailor-4G";
6 const char* password = "Nailarahma";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 11;
12 DHT dht(dhtPin, DHT22);
13
14 const char index_html[] PROGMEM = R"rawliteral{
15 <!DOCTYPE html>
16 <html>
17 <head>
18   <meta name="viewport" content="width=device-width, initial-scale=1">
```

Output Serial Monitor: X

Message (Enter to send message to "NodeMCU 1.0 (ESP-12B Module)" on "COM6")

IP Address Server Anda:
192.168.18.53



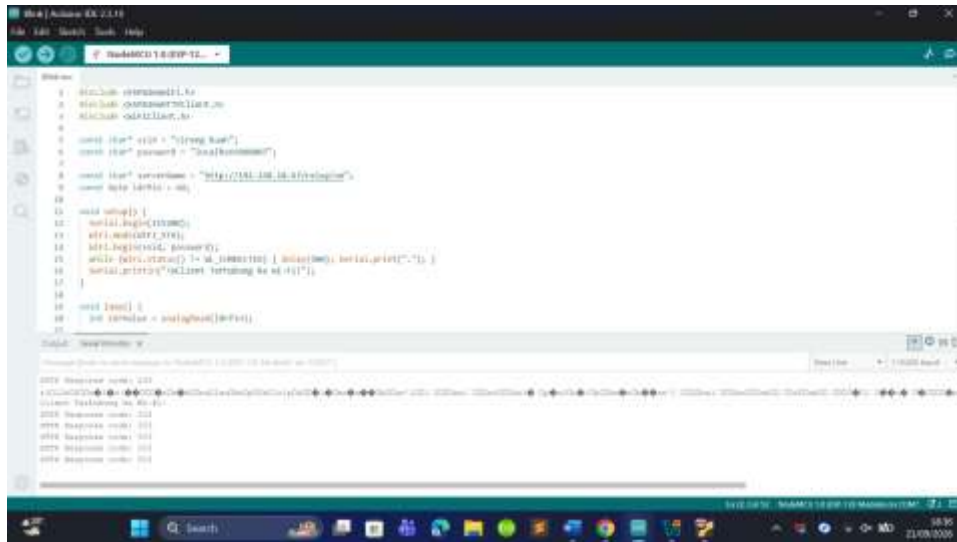
3.5.2 HTTP Client

Pada praktikum 3.5.2, ESP8266 digunakan sebagai **Client** yang bertugas mengirimkan permintaan HTTP kepada Web Server. Client terhubung ke jaringan Wi-Fi yang sama dengan Server agar dapat mengakses alamat IP Server.

Pada percobaan awal, Client menggunakan nilai sensor sebagai kondisi untuk menentukan kapan HTTP GET dikirimkan. Ketika kondisi yang ditentukan terpenuhi, program membuat koneksi ke Server dan mengirimkan HTTP GET menggunakan fungsi `http.GET()`.

HTTP GET digunakan untuk meminta Server mengakses alamat tertentu. Pada praktikum ini, alamat tersebut mengarah ke endpoint yang telah dibuat pada Server, sehingga Server dapat menjalankan perintah untuk mengontrol LED.

Hasil komunikasi antara Client dan Server dapat dilihat melalui **HTTP Response Code** pada Serial Monitor. Jika permintaan berhasil diterima oleh Server, Client akan mendapatkan kode respons HTTP dari Server.



```
1 #include <ESP8266.h>
2 #include <WiFi.h>
3 #include <HTTP.h>
4
5 const char* ssid = "String Ssid";
6 const char* password = "Password";
7
8 const char* serverName = "http://192.168.1.1:80/";
9 const byte serverPort = 80;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.begin(ssid, password);
14   while (WiFi.status() != WL_CONNECTED) { delay(1000); Serial.print("."); }
15   Serial.println("ESP8266 terhubung ke Wi-Fi");
16 }
17
18 void loop() {
19   HTTPServer server(serverPort);
20 }
```

Serial Monitor Output:

```
2019-09-20 10:00:00: HTTPServer started
2019-09-20 10:00:00: ESP8266 connected to Wi-Fi
2019-09-20 10:00:00: HTTPServer started
2019-09-20 10:00:00: ESP8266 connected to Wi-Fi
2019-09-20 10:00:00: HTTPServer started
2019-09-20 10:00:00: ESP8266 connected to Wi-Fi
2019-09-20 10:00:00: HTTPServer started
2019-09-20 10:00:00: ESP8266 connected to Wi-Fi
```



3.6 Latihan

Latihan 1 — Penggunaan Mode STA

Mode **STA (Station)** digunakan karena ESP8266 pada praktikum ini perlu terhubung ke jaringan Wi-Fi yang sudah tersedia. Dengan mode tersebut, ESP8266 dapat menjadi bagian dari jaringan yang sama dengan perangkat lainnya.

Penggunaan STA memudahkan komunikasi antara Client dan Server karena keduanya dapat berada dalam satu jaringan Wi-Fi. Jika menggunakan mode AP, ESP8266 akan membuat jaringan

Wi-Fi sendiri, sedangkan pada praktikum ini komunikasi dilakukan melalui jaringan yang sudah tersedia.

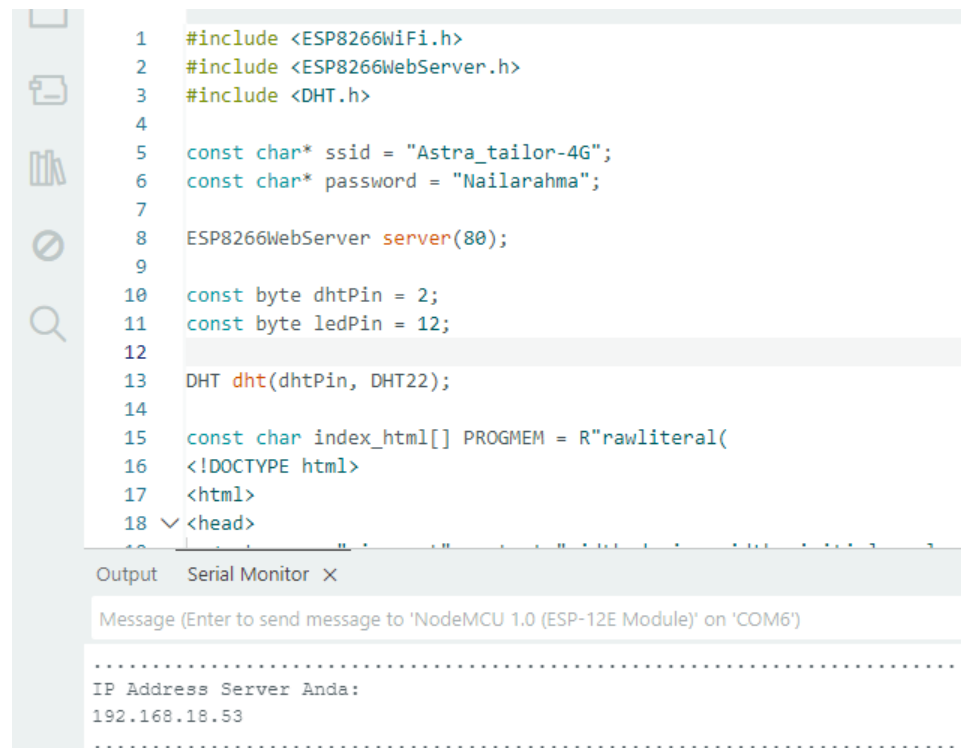
Latihan 2 — Penambahan Kelembapan dan Tombol Dinamis

Pada latihan ini, program Web Server dimodifikasi dengan menambahkan informasi **kelembapan** yang dibaca menggunakan sensor DHT22. Selain suhu, DHT22 dapat memberikan nilai kelembapan melalui fungsi `readHumidity()`.

Hasil pembacaan suhu dan kelembapan kemudian ditampilkan pada halaman Web Server. Dengan adanya penambahan tersebut, pengguna dapat melihat dua informasi dari sensor secara langsung melalui browser.

Selain itu, dua tombol ON dan OFF yang sebelumnya tersedia diubah menjadi **satu tombol dinamis**. Tombol akan menyesuaikan dengan kondisi LED saat ini. Jika LED dalam keadaan mati, halaman akan menampilkan tombol **“NYALAKAN (ON)”**. Jika LED dalam keadaan menyala, halaman akan menampilkan tombol **“MATIKAN (OFF)”**.

Dengan perubahan tersebut, tampilan Web Server menjadi lebih sederhana karena hanya menampilkan tombol yang sesuai dengan kondisi LED saat itu.

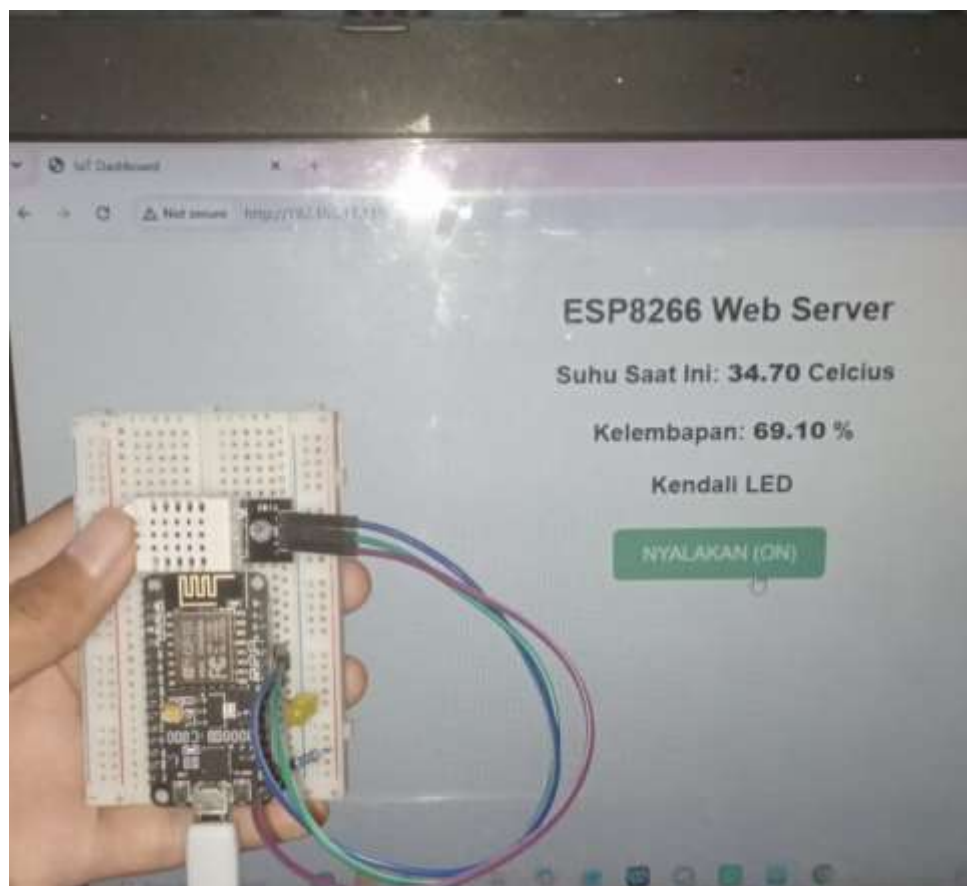
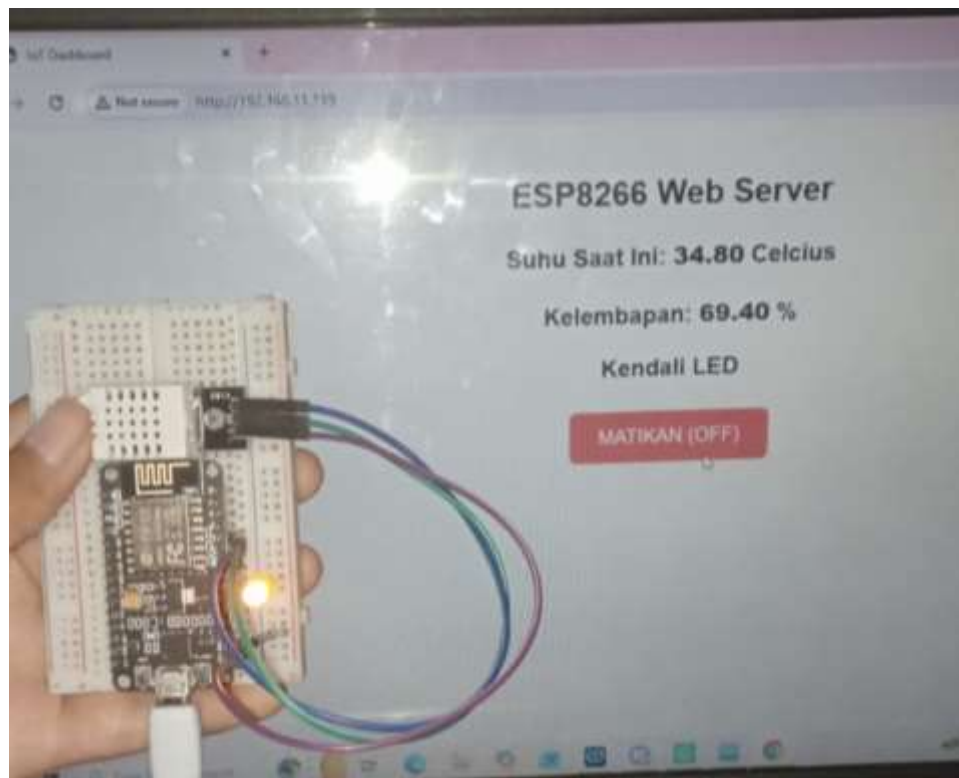


```
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "Astra_tailor-4G";
6  const char* password = "Nailarahma";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte ledPin = 12;
12
13 DHT dht(dhtPin, DHT22);
14
15 const char index_html[] PROGMEM = R"rawliteral(
16 <!DOCTYPE html>
17 <html>
18 <head>
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')

```
.....
IP Address Server Anda:
192.168.18.53
.....
```



Latihan 3 — Kondisi Ketika Server Tidak Tersedia

Latihan ini digunakan untuk mengetahui kondisi yang terjadi ketika Client mencoba mengirim HTTP GET tetapi Server tidak dapat diakses.

Pada kondisi normal, fungsi `http.GET()` akan menerima HTTP Response Code dari Server. Namun, apabila Server mati atau tidak dapat dihubungi, Client tidak akan mendapatkan respons HTTP normal. Nilai yang diperoleh dapat berupa **kode negatif**, misalnya -1, yang menunjukkan bahwa proses komunikasi mengalami kegagalan.

Hal tersebut menunjukkan bahwa kode negatif pada `http.GET()` berbeda dengan HTTP status code seperti 200. Kode 200 merupakan respons dari Server yang menunjukkan bahwa permintaan berhasil, sedangkan kode negatif menunjukkan adanya masalah dalam proses koneksi atau komunikasi dengan Server.

Latihan 4 — Pengiriman GET Berdasarkan Suhu

Pada latihan ini, kondisi pengiriman HTTP GET diubah dari penggunaan LDR menjadi berdasarkan **suhu yang dibaca oleh DHT22**.

Client membaca suhu menggunakan fungsi `readTemperature()`. Setelah mendapatkan nilai suhu, program melakukan pengecekan terhadap batas yang telah ditentukan, yaitu **35°C**.

Jika suhu lebih dari 35°C dan Client masih terhubung ke Wi-Fi, Client akan mengirimkan HTTP GET kepada Server. Jika suhu 35°C atau kurang, Client tidak mengirimkan HTTP GET.

Dengan demikian, pengiriman data dari Client tidak dilakukan secara terus-menerus, tetapi hanya dilakukan ketika kondisi suhu yang telah ditentukan terpenuhi.

3.7 Auto Refresh Web Server

Pada praktikum 3.7, Web Server dimodifikasi agar halaman web dapat **diperbarui secara otomatis setiap 5 detik**. Modifikasi dilakukan dengan menambahkan perintah meta refresh pada bagian HTML halaman Web Server.

Dengan adanya fitur tersebut, browser akan melakukan refresh secara otomatis setiap lima detik. Setiap kali halaman diperbarui, ESP8266 akan membaca kembali data dari DHT22 sehingga nilai suhu yang ditampilkan dapat mengikuti perubahan kondisi sensor.

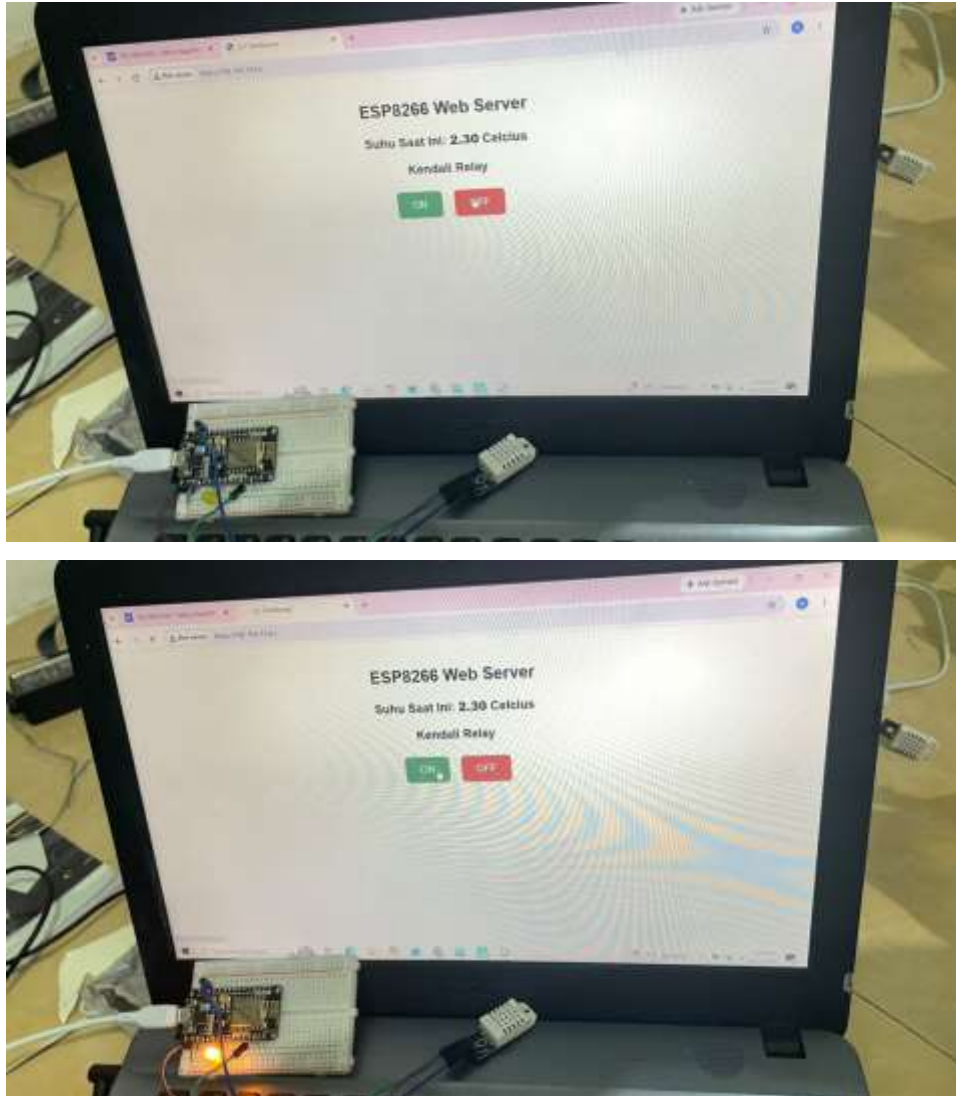
Penggunaan auto refresh membuat pengguna tidak perlu melakukan refresh halaman secara manual untuk melihat data terbaru. Namun, metode ini juga menyebabkan browser mengirimkan

permintaan kembali secara berkala. Jika terdapat banyak Client yang melakukan refresh dalam waktu yang sama, jumlah request yang diterima Server dapat meningkat.

Selain itu, proses refresh dapat sedikit mengganggu interaksi pengguna apabila dilakukan bersamaan dengan aktivitas pada halaman web.



```
3.7.no
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "Astra_tallor-4G";
6 const char* password = "Mallarama";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte ledPin = 12;
12
13 DHT dht(dhtPin, DHT22);
14
15 const char index_html[] PROGMEM = R"rawliteral(
16 <!DOCTYPE html>
17 <html>
18 <head>
```



Kesimpulan Pembahasan

Berdasarkan praktikum yang dilakukan, ESP8266 dapat digunakan sebagai **Web Server** maupun **HTTP Client** untuk melakukan komunikasi melalui jaringan Wi-Fi. Web Server dapat menampilkan data suhu dan kelembapan dari DHT22 serta mengendalikan LED melalui browser, sedangkan Client dapat mengirimkan HTTP GET berdasarkan kondisi tertentu.

Pada latihan pengembangan, sistem dibuat lebih informatif dengan menambahkan kelembapan, membuat tombol LED menjadi dinamis, menganalisis kondisi ketika Server tidak tersedia, serta menggunakan suhu sebagai kondisi pengiriman HTTP GET. Pada praktikum 3.7, halaman Web Server juga dibuat dapat diperbarui secara otomatis setiap lima detik sehingga data sensor dapat dipantau secara berkala.

